Week-1

Name- karthik kumar c
Deep Skilling Course-Java FSE -React

Engineering Concepts: Design Patterns and Principles

Exercise 1: Implementing the Singleton Pattern

Scenario:

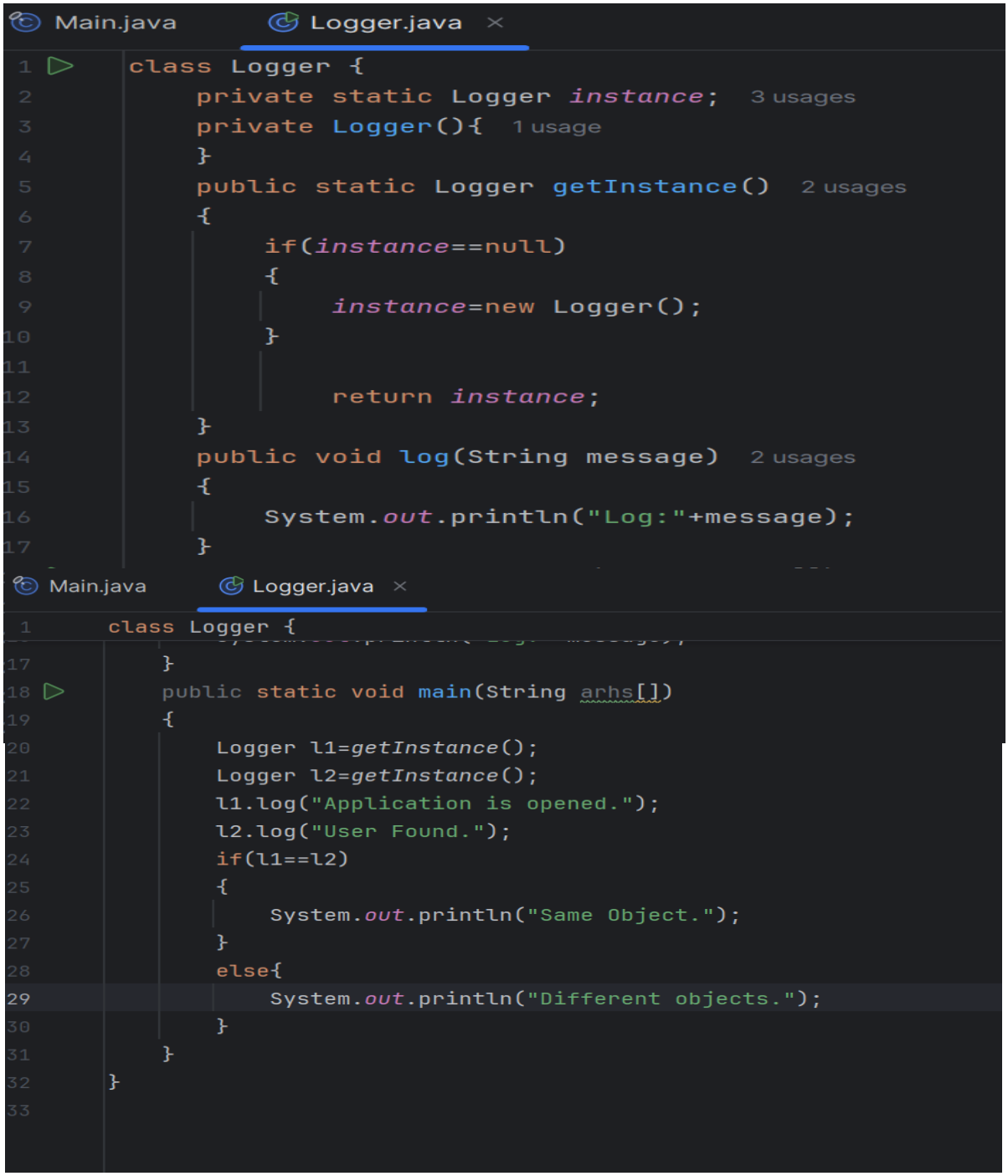
You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

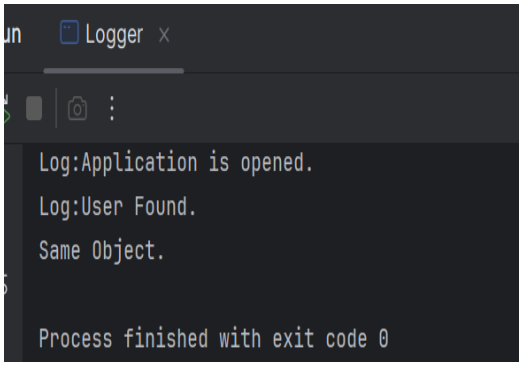
Steps:

1. **Create a New Java Project:**
    - Create a new Java project named **SingletonPatternExample**.
2. **Define a Singleton Class:**
    - Create a class named Logger that has a private static instance of itself.
    - Ensure the constructor of Logger is private.
    - Provide a public static method to get the instance of the Logger class.
3. **Implement the Singleton Pattern:**
    - Write code to ensure that the Logger class follows the Singleton design pattern.
4. **Test the Singleton Implementation:**

- Create a test class to verify that only one instance of Logger is created and used across the application.

Output:





Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

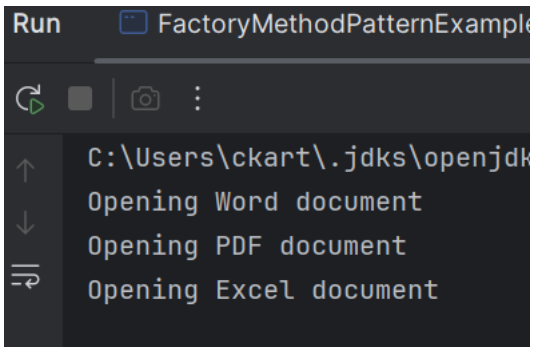
1. **Create a New Java Project:**
    - Create a new Java project named **FactoryMethodPatternExample**.
2. **Define Document Classes:**
    - Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.
3. **Create Concrete Document Classes:**
    - Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.
4. **Implement the Factory Method:**
    - Create an abstract class **DocumentFactory** with a method **createDocument()**.
    - Create concrete factory classes for each document type that extends DocumentFactory and implements the **createDocument()** method.
5. **Test the Factory Method Implementation:**
    - Create a test class to demonstrate the creation of different document types using the factory method.

Code:

```
interface Document {
   void open();
}

class WordDocument implements Document {
   public void open() {
      System.out.println("Opening Word document");
   }
}

class PdfDocument implements Document {
   public void open() {
      System.out.println("Opening PDF document");
   }
}

class ExcelDocument implements Document {
   public void open() {
      System.out.println("Opening Excel document");
   }
}

abstract class DocumentFactory {
   abstract Document createDocument();
}

class WordFactory extends DocumentFactory {
   Document createDocument() {
      return new WordDocument();
   }
}

class PdfFactory extends DocumentFactory {
   Document createDocument() {
      return new PdfDocument();
   }
}

class ExcelFactory extends DocumentFactory {
```

```
    Document createDocument() {
        return new ExcelDocument();
    }
}
```

output:



Exercise 3: Implementing the Builder Pattern

Scenario:

You are developing a system to create complex objects such as a Computer with multiple optional parts. Use the Builder Pattern to manage the construction process.

Steps:

1. **Create a New Java Project:**
    - Create a new Java project named **BuilderPatternExample**.
2. **Define a Product Class:**
    - Create a class **Computer** with attributes like **CPU**, **RAM**, **Storage**, etc.
3. **Implement the Builder Class:**
    - Create a static nested Builder class inside Computer with methods to set each attribute.
    - Provide a **build()** method in the Builder class that returns an instance of Computer.
4. **Implement the Builder Pattern:**
    - Ensure that the **Computer** class has a private constructor that takes the **Builder** as a parameter.
5. **Test the Builder Implementation:**

- Create a test class to demonstrate the creation of different configurations of Computer using the Builder pattern.

Code:

```
class Computer {
    private String CPU;
    private String RAM;
    private String storage;
    private String graphicsCard;

    private Computer(Builder builder) {
        this.CPU = builder.CPU;
        this.RAM = builder.RAM;
        this.storage = builder.storage;
        this.graphicsCard = builder.graphicsCard;
    }

    public void displayDetails() {
        System.out.println("CPU: " + CPU);
        System.out.println("RAM: " + RAM);
        System.out.println("Storage: " + storage);
        System.out.println("Graphics Card: " + graphicsCard);
        System.out.println();
    }

    static class Builder {
        private String CPU;
        private String RAM;
        private String storage;
        private String graphicsCard;

        public Builder setCPU(String CPU) {
            this.CPU = CPU;
            return this;
        }

        public Builder setRAM(String RAM) {
            this.RAM = RAM;
            return this;
        }

        public Builder setStorage(String storage) {
            this.storage = storage;
            return this;
        }

        public Builder setGraphicsCard(String graphicsCard) {
            this.graphicsCard = graphicsCard;
            return this;
        }

        public Computer build() {
            return new Computer(this);
        }
    }
}

public class BuilderPatternExample {
    public static void main(String[] args) {
```

```
        Computer gamingComputer = new Computer.Builder()
                .setCPU("Intel i7")
                .setRAM("16GB")
                .setStorage("1TB SSD")
                .setGraphicsCard("NVIDIA RTX 4060")
                .build();

        Computer officeComputer = new Computer.Builder()
                .setCPU("Intel i5")
                .setRAM("8GB")
                .setStorage("512GB SSD")
                .build();

        gamingComputer.displayDetails();
        officeComputer.displayDetails();
    }
}
```

output:

CPU: Intel i7

RAM: 16GB

Storage: 1TB SSD

Graphics Card: NVIDIA RTX 4060

CPU: Intel i5

RAM: 8GB

Storage: 512GB SSD

Graphics Card: null

Exercise 4: Implementing the Adapter Pattern

Scenario:

You are developing a payment processing system that needs to integrate with multiple third-party payment gateways with different interfaces. Use the Adapter Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
    - Create a new Java project named **AdapterPatternExample**.
2. **Define Target Interface:**
    - Create an interface **PaymentProcessor** with methods like **processPayment()**.
3. **Implement Adaptee Classes:**

- Create classes for different payment gateways with their own methods.

4. **Implement the Adapter Class:**
   - Create an adapter class for each payment gateway that implements PaymentProcessor and translates the calls to the gateway-specific methods.
5. **Test the Adapter Implementation:**
   - Create a test class to demonstrate the use of different payment gateways through the adapter.

Code:

```
interface PaymentProcessor {
    void processPayment(double amount);
}

class PaymentGateway1 {
    void paymentGateway1Payment(double amount) {
        System.out.println("Payment processed using PaymentGateway1: " +
amount);
    }
}

class PaymentGateway2 {
    void paymentGateway2Payment(double amount) {
        System.out.println("Payment processed using PaymentGateway2: " +
amount);
    }
}

class PaymentGateway1Adapter implements PaymentProcessor {
    private PaymentGateway1 paymentGateway1;

    PaymentGateway1Adapter(PaymentGateway1 paymentGateway1) {
        this.paymentGateway1 = paymentGateway1;
    }

    public void processPayment(double amount) {
        paymentGateway1.paymentGateway1Payment(amount);
    }
}

class PaymentGateway2Adapter implements PaymentProcessor {
    private PaymentGateway2 paymentGateway2;

    PaymentGateway2Adapter(PaymentGateway2 paymentGateway2) {
        this.paymentGateway2 = paymentGateway2;
    }

    public void processPayment(double amount) {
        paymentGateway2.paymentGateway2Payment(amount);
    }
}

public class AdapterPatternExample {
    public static void main(String[] args) {
```

```
        PaymentProcessor paymentProcessor1 =
                new PaymentGateway1Adapter(new PaymentGateway1());

        PaymentProcessor paymentProcessor2 =
                new PaymentGateway2Adapter(new PaymentGateway2());

        paymentProcessor1.processPayment(1000);
        paymentProcessor2.processPayment(2000);
    }
}
```

Output:

Payment processed using PaymentGateway1: 1000.0

Payment processed using PaymentGateway2: 2000.0

Exercise 5: Implementing the Decorator Pattern

Scenario:

You are developing a notification system where notifications can be sent via multiple channels (e.g., Email, SMS). Use the Decorator Pattern to add functionalities dynamically.

Steps:

1. **Create a New Java Project:**
    - Create a new Java project named **DecoratorPatternExample**.
2. **Define Component Interface:**
    - Create an interface **Notifier** with a method **send()**.
3. **Implement Concrete Component:**
    - Create a class **EmailNotifier** that implements Notifier.
4. **Implement Decorator Classes:**
    - Create abstract decorator class **NotifierDecorator** that implements **Notifier** and holds a reference to a **Notifier** object.
    - Create concrete decorator classes like **SMSNotifierDecorator**, **SlackNotifierDecorator** that extend **NotifierDecorator**.
5. **Test the Decorator Implementation:**
    - Create a test class to demonstrate sending notifications via multiple channels using decorators.

Code:

```
interface Notifier {
    void send(String message);
}

class EmailNotifier implements Notifier {
    public void send(String message) {
        System.out.println("Email Notification: " + message);
    }
}

abstract class NotifierDecorator implements Notifier {
    protected Notifier notifier;

    NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }

    public void send(String message) {
        notifier.send(message);
    }
}

class SMSNotifierDecorator extends NotifierDecorator {
    SMSNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    public void send(String message) {
        super.send(message);
        System.out.println("SMS Notification: " + message);
    }
}

class SlackNotifierDecorator extends NotifierDecorator {
    SlackNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    public void send(String message) {
        super.send(message);
        System.out.println("Slack Notification: " + message);
    }
}

public class DecoratorPatternExample {
    public static void main(String[] args) {
        Notifier notifier = new EmailNotifier();

        notifier = new SMSNotifierDecorator(notifier);
        notifier = new SlackNotifierDecorator(notifier);

        notifier.send("Notification sent successfully");
    }
}
```

Output:

Email Notification: Notification sent successfully

SMS Notification: Notification sent successfully

Slack Notification: Notification sent successfully

Exercise 6: Implementing the Proxy Pattern

Scenario:

You are developing an image viewer application that loads images from a remote server. Use the Proxy Pattern to add lazy initialization and caching.

Steps:

1. **Create a New Java Project:**
    - Create a new Java project named **ProxyPatternExample**.
2. **Define Subject Interface:**
    - Create an interface Image with a method **display()**.
3. **Implement Real Subject Class:**
    - Create a class **RealImage** that implements Image and loads an image from a remote server.
4. **Implement Proxy Class:**
    - Create a class **ProxyImage** that implements Image and holds a reference to RealImage.
    - Implement lazy initialization and caching in **ProxyImage**.
5. **Test the Proxy Implementation:**
    - Create a test class to demonstrate the use of **ProxyImage** to load and display images.

Code:

```
interface Image {
    void display();
}

class RealImage implements Image {
    private String fileName;

    RealImage(String fileName) {
        this.fileName = fileName;
        loadFromRemoteServer();
    }

    private void loadFromRemoteServer() {
        System.out.println("Loading image from remote server: " +
fileName);
    }

    public void display() {
        System.out.println("Displaying image: " + fileName);
    }
}

class ProxyImage implements Image {
    private RealImage realImage;
    private String fileName;

    ProxyImage(String fileName) {
        this.fileName = fileName;
    }

    public void display() {
        if (realImage == null) {
            realImage = new RealImage(fileName);
        }
        realImage.display();
    }
}

public class ProxyPatternExample {
    public static void main(String[] args) {
        Image image = new ProxyImage("photo.jpg");

        image.display();
        image.display();
    }
}
```

Output:

Loading image from remote server: photo.jpg

Displaying image: photo.jpg

Displaying image: photo.jpg

Exercise 7: Implementing the Observer Pattern

Scenario:

You are developing a stock market monitoring application where multiple clients need to be notified whenever stock prices change. Use the Observer Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
    - Create a new Java project named **ObserverPatternExample**.
2. **Define Subject Interface:**
    - Create an interface **Stock** with methods to **register**, **deregister**, and **notify** observers.
3. **Implement Concrete Subject:**
    - Create a class **StockMarket** that implements **Stock** and maintains a list of observers.
4. **Define Observer Interface:**
    - Create an interface Observer with a method **update().**
5. **Implement Concrete Observers:**
    - Create classes **MobileApp**, **WebApp** that implement Observer.
6. **Test the Observer Implementation:**
    - Create a test class to demonstrate the registration and notification of observers.

Code:

```
import java.util.ArrayList;
import java.util.List;

interface Stock {
    void register(Observer observer);
    void deregister(Observer observer);
    void notifyObservers();
}

interface Observer {
    void update(double price);
}

class StockMarket implements Stock {
    private List<Observer> observers = new ArrayList<>();
    private double price;

    public void register(Observer observer) {
        observers.add(observer);
    }

    public void deregister(Observer observer) {
        observers.remove(observer);
    }

    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(price);
        }
    }

    public void setPrice(double price) {
        this.price = price;
        notifyObservers();
    }
}

class MobileApp implements Observer {
    public void update(double price) {
        System.out.println("MobileApp notified. Stock price: " + price);
    }
}

class WebApp implements Observer {
    public void update(double price) {
        System.out.println("WebApp notified. Stock price: " + price);
    }
}

public class ObserverPatternExample {
    public static void main(String[] args) {
        StockMarket stockMarket = new StockMarket();

        Observer mobileApp = new MobileApp();
        Observer webApp = new WebApp();

        stockMarket.register(mobileApp);
        stockMarket.register(webApp);
```

```
        stockMarket.setPrice(1500.50);
    }
}
```

Output:

MobileApp notified. Stock price: 1500.5

WebApp notified. Stock price: 1500.5

Exercise 8: Implementing the Strategy Pattern

Scenario:

You are developing a payment system where different payment methods (e.g., Credit Card, PayPal) can be selected at runtime. Use the Strategy Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
    - Create a new Java project named **StrategyPatternExample**.
2. **Define Strategy Interface:**
    - Create an interface PaymentStrategy with a method **pay()**.
3. **Implement Concrete Strategies:**
    - Create classes **CreditCardPayment**, **PayPalPayment** that implement **PaymentStrategy**.
4. **Implement Context Class:**
    - Create a class **PaymentContext** that holds a reference to **PaymentStrategy** and a method to execute the strategy.
5. **Test the Strategy Implementation:**

Create a test class to demonstrate selecting and using different payment strategies

Code:

```
interface PaymentStrategy {
    void pay(double amount);
}

class CreditCardPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid using Credit Card: " + amount);
    }
}

class PayPalPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid using PayPal: " + amount);
    }
}

class PaymentContext {
    private PaymentStrategy paymentStrategy;

    PaymentContext(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

    public void executeStrategy(double amount) {
        paymentStrategy.pay(amount);
    }
}

public class StrategyPatternExample {
    public static void main(String[] args) {
        PaymentContext creditCardPayment =
                new PaymentContext(new CreditCardPayment());
        creditCardPayment.executeStrategy(1000);

        PaymentContext payPalPayment =
                new PaymentContext(new PayPalPayment());
        payPalPayment.executeStrategy(2000);
    }
}
```

Output:

Paid using Credit Card: 1000.0

Paid using PayPal: 2000.0

Exercise 9: Implementing the Command Pattern

Scenario: You are developing a home automation system where commands can be issued to turn devices on or off. Use the Command Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
    - Create a new Java project named **CommandPatternExample**.
2. **Define Command Interface:**
    - Create an interface Command with a method **execute()**.
3. **Implement Concrete Commands:**
    - Create classes **LightOnCommand**, **LightOffCommand** that implement Command.
4. **Implement Invoker Class:**
    - Create a class **RemoteControl** that holds a reference to a Command and a method to execute the command.
5. **Implement Receiver Class:**
    - Create a class **Light** with methods to turn on and off.
6. **Test the Command Implementation:**
    - Create a test class to demonstrate issuing commands using the **RemoteControl**.

Exercise 10: Implementing the MVC Pattern

Scenario:

You are developing a simple web application for managing student records using the MVC pattern.

Code:

```
interface Command {
    void execute();
}

class Light {
    public void turnOn() {
        System.out.println("Light is ON");
    }

    public void turnOff() {
        System.out.println("Light is OFF");
    }
}

class LightOnCommand implements Command {
    private Light light;

    LightOnCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.turnOn();
    }
}

class LightOffCommand implements Command {
    private Light light;

    LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.turnOff();
    }
}

class RemoteControl {
    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public void executeCommand() {
        command.execute();
    }
}

public class CommandPatternExample {
    public static void main(String[] args) {
        Light light = new Light();

        Command lightOn = new LightOnCommand(light);
        Command lightOff = new LightOffCommand(light);
```

```
        RemoteControl remoteControl = new RemoteControl();

        remoteControl.setCommand(lightOn);
        remoteControl.executeCommand();

        remoteControl.setCommand(lightOff);
        remoteControl.executeCommand();
    }
}
```

Output:

Light is ON

Light is OFF

Exercise 10: Implementing the MVC Pattern

Scenario:

You are developing a simple web application for managing student records using the MVC pattern.

Steps:

1. **Create a New Java Project:**
    - Create a new Java project named **MVCPatternExample**.
2. **Define Model Class:**
    - Create a class **Student** with attributes like **name, id, and grade**.
3. **Define View Class:**
    - Create a class **StudentView** with a method **displayStudentDetails()**.
4. **Define Controller Class:**
    - Create a class **StudentController** that handles the communication between the model and the view.
5. **Test the MVC Implementation:**
    - Create a main class to demonstrate creating a **Student**, updating its details using **StudentController**, and displaying them using **StudentView**.

Code:

```
class Student {
    private String name;
    private int id;
    private String grade;

    Student(String name, int id, String grade) {
        this.name = name;
        this.id = id;
        this.grade = grade;
    }

    public String getName() {
        return name;
    }

    public int getId() {
        return id;
    }

    public String getGrade() {
        return grade;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setGrade(String grade) {
        this.grade = grade;
    }
}

class StudentView {
    public void displayStudentDetails(String name, int id, String grade) {
        System.out.println("Student Name: " + name);
        System.out.println("Student ID: " + id);
        System.out.println("Student Grade: " + grade);
    }
}

class StudentController {
    private Student student;
    private StudentView studentView;

    StudentController(Student student, StudentView studentView) {
        this.student = student;
        this.studentView = studentView;
    }

    public void setStudentName(String name) {
        student.setName(name);
    }

    public void setStudentGrade(String grade) {
        student.setGrade(grade);
    }
```

```
    public void updateView() {
        studentView.displayStudentDetails(
                student.getName(),
                student.getId(),
                student.getGrade()
        );
    }
}

public class MVCPatternExample {
    public static void main(String[] args) {
        Student student = new Student("Karthik", 101, "A");
        StudentView studentView = new StudentView();

        StudentController studentController =
                new StudentController(student, studentView);

        studentController.updateView();

        studentController.setStudentName("Kumar");
        studentController.setStudentGrade("A+");

        studentController.updateView();
    }
}
```

Output:

Student Name: Karthik

Student ID: 101

Student Grade: A

Student Name: Kumar

Student ID: 101

Student Grade: A+

Exercise 11: Implementing Dependency Injection

Scenario:

You are developing a customer management application where the service class depends on a repository class. Use Dependency Injection to manage these dependencies.

Steps:

1. **Create a New Java Project:**

- Create a new Java project named **DependencyInjectionExample**.

2. **Define Repository Interface:**
   - Create an interface **CustomerRepository** with methods like **findCustomerById()**.
3. **Implement Concrete Repository:**
   - Create a class **CustomerRepositoryImpl** that implements **CustomerRepository**.
4. **Define Service Class:**
   - Create a class **CustomerService** that depends on **CustomerRepository**.
5. **Implement Dependency Injection:**
   - Use constructor injection to inject **CustomerRepository** into **CustomerService**.
6. **Test the Dependency Injection Implementation:**
   - Create a main class to demonstrate creating a **CustomerService** with **CustomerRepositoryImpl** and using it to find a customer.

Code:

```
interface CustomerRepository {
    String findCustomerById(int id);
}

class CustomerRepositoryImpl implements CustomerRepository {
    public String findCustomerById(int id) {
        return "Customer found with ID: " + id;
    }
}

class CustomerService {
    private CustomerRepository customerRepository;

    CustomerService(CustomerRepository customerRepository) {
        this.customerRepository = customerRepository;
    }

    public void findCustomer(int id) {
        System.out.println(customerRepository.findCustomerById(id));
    }
}

public class DependencyInjectionExample {
    public static void main(String[] args) {
        CustomerRepository customerRepository = new
CustomerRepositoryImpl();

        CustomerService customerService =
                new CustomerService(customerRepository);

        customerService.findCustomer(101);
```

```
        }
}
```

Output:

Customer found with ID: 101